
Eduardo Gomes Martins e Lucas de Lima Bergami

Recomendação de Repositórios no GitHub: Uma Abordagem Baseada em Redes Sociais e Aprendizado de Máquina

1 Contexto

O desenvolvimento de software moderno é, em sua essência, uma atividade coletiva. Plataformas como o GitHub reúnem milhões de desenvolvedores em torno de projetos de código aberto, criando um ecossistema em que o ato técnico de programar se entrelaça com dinâmicas sociais bem definidas: seguir outros profissionais, estreitar projetos, reportar problemas e, principalmente, contribuir diretamente para repositórios alheios por meio de *commits* e *pull requests*. Esse arranjo posiciona o GitHub como uma rede social de natureza heterogênea, ao mesmo tempo plataforma de versionamento de código e comunidade de prática.

A pergunta central que emerge desse cenário é: *o que leva um desenvolvedor a contribuir em um repositório específico?* Fatores técnicos como compatibilidade de linguagem e interesses temáticos convivem com fatores sociais como pertencer ao círculo de pessoas que o criador do projeto já segue. Compreender e quantificar essa dinâmica tem aplicação direta na recomendação de projetos, na formação de equipes e na análise da difusão de conhecimento técnico em comunidades de código aberto.

2 Objetivo

Este trabalho propõe o desenvolvimento de um sistema de recomendação de repositórios baseado em análise de redes sociais e aprendizado de máquina. O objetivo central é estimar a probabilidade de que um determinado usuário venha a contribuir em um repositório específico que ainda não faz parte do seu histórico de colaboração.

Em termos formais, o problema é tratado como uma tarefa de *link prediction* em um grafo heterogêneo: dada a rede de relações existente, prever quais arestas do tipo “contribui” surgirão entre pares (usuário, repositório) ainda não conectados. A saída prática do sistema é uma lista ordenada de repositórios candidatos para um dado desenvolvedor, ranqueados pela probabilidade estimada de contribuição. Busca-se também investigar como relações sociais prévias, proximidade estrutural na rede e características intrínsecas dos projetos influenciam a formação dessas novas conexões.

3 Metodologia

3.1 Parte 1: Descrição dos dados e modelagem

3.1.1 Extração dos dados

A base de dados foi coletada diretamente da API pública do GitHub (`api.github.com`), utilizando um script Python desenvolvido especificamente para este trabalho. A estratégia de descoberta de usuários seguiu uma busca em largura (*BFS*) a partir de cinco perfis-semente (*torvalds*, *karpthy*, *tensorflow*, *pytorch*, *huggingface*), expandindo a fronteira por meio da relação de seguidores de cada nó visitado até atingir o limite configurado. Para cada usuário descoberto, foram coletados dados de perfil (contagem de seguidores, número de eventos recentes por tipo, idade da conta, linguagens utilizadas, presença de *README* pessoal), bem como seus repositórios mais relevantes por número de estrelas.

Para cada repositório, extraíram-se estatísticas (estrelas, *forks*, *watchers*, *issues* abertas, tamanho), metadados (linguagem primária, tópicos, licença, data do último *push*) e o conteúdo textual do arquivo *README*. Adicionalmente, o *endpoint* de contribuidores de cada repositório foi consultado para identificar quais usuários já haviam colaborado naquele projeto, informação essa que define os rótulos do problema supervisionado.

As respostas da API, no formato JSON, foram limpas e persistidas em arquivos CSV separados para usuários, repositórios e três tabelas de arestas (seguir, possuir, contribuir). Um mecanismo de *checkpoint* em JSON foi implementado para preservar o progresso em caso de interrupção por limite de requisições (*rate limit*). Para acelerar a coleta, chamadas independentes de cada usuário (eventos, repositórios, lista de seguidos, *README*) foram paralelizadas via `ThreadPoolExecutor`.

O escopo dos dados exclui o conteúdo dos *commits* em si (o código produzido) e dados privados; mantém-se apenas o registro de que a contribuição existiu e os metadados públicos disponíveis pela API. Ao término da coleta, o conjunto consolidado totalizou **4.999 usuários** e **23.118 repositórios** únicos.

3.1.2 Modelagem da rede

A rede foi concebida como um grafo heterogêneo com dois tipos de vértices: **Usuários** e **Repositórios**. As arestas representam três relações observáveis distintas:

- **Segue:** aresta direcionada de um Usuário para outro Usuário, extraída da lista de followers following da API.
- **Possui:** aresta direcionada de um Usuário para um Repositório do qual ele é criador.
- **Contribui:** aresta direcionada de um Usuário para um Repositório de terceiros no qual ele figura como contribuidor, extraída do *endpoint* /contributors da API.

Optou-se por implementar um *multigrafo* dirigido em vez de um grafo simples. Essa escolha foi motivada pelo fato de que um mesmo par usuário-repositório pode abrigar mais de uma

relação simultaneamente: o caso mais comum é o criador de um projeto aparecer tanto como possuidor quanto como colaborador de seu próprio repositório. Um grafo simples sobrescreveria uma das relações, com perda de informação. O grafo final foi exportado no formato GraphML para análise posterior.

3.2 Parte 2: Experimentos

3.2.1 Análise descritiva básica

O grafo construído contém **137.987 nós** e **210.588 arestas**, distribuídos conforme a Tabela 1.

Tabela 1: Estatísticas básicas da rede heterogênea construída.

Métrica	Valor
Nós totais	137.987
Usuários	4.999
Repositórios	23.118
Outros nós*	109.870
Arestas totais	210.588
segue	8,254
possui	11,973
contribui	190,361
Densidade	$\approx 1,1 \times 10^{-5}$

* Nós de repositórios e usuários descobertos indiretamente via contribuições.

A densidade extremamente baixa é esperada e característica de redes de colaboração em larga escala: cada desenvolvedor interage diretamente com uma fração ínfima dos repositórios existentes na plataforma, e a rede resultante é esparsa por construção. Esse fenômeno motiva diretamente a necessidade de um sistema de recomendação, justamente porque a maioria das conexões potencialmente úteis ainda não foi formada.

A distribuição de graus apresenta cauda pesada (*heavy tail*), característica de redes livres de escala: a maioria dos usuários possui poucos seguidores e contribui em poucos repositórios, enquanto um número reduzido de perfis concentra graus de entrada muito elevados. Esse padrão é comum em ecossistemas de código aberto, onde projetos de grande popularidade (como *linux*, *tensorflow* ou *rails*) atuam como atratores de contribuição para a comunidade.

3.2.2 Análise topológica

A análise topológica da rede evidencia uma estrutura altamente esparsa, porém completamente conectada. O grafo possui uma única componente conexa contendo todos os 137.987 nós, indicando que qualquer usuário ou repositório pode ser alcançado a partir de outro por algum caminho. Essa característica favorece o uso de medidas de proximidade na recomendação, permitindo explorar relações indiretas entre desenvolvedores e projetos.

A distribuição de graus apresenta forte assimetria, típica de redes complexas. Enquanto a média de grau é de apenas 3,05 (mediana igual a 1), alguns poucos nós concentram um número extremamente elevado de conexões, como `dependabot` [bot] (2.271 arestas) e `github-actions` [bot] (780 arestas). Esse comportamento caracteriza uma distribuição de cauda longa (*long-tail*), na qual a maioria dos nós possui poucas conexões e um pequeno conjunto de vértices atua como hubs da rede.

O coeficiente médio de clustering foi de apenas 0,0332, tanto para o grafo completo quanto para o subgrafo formado apenas por usuários ligados pela relação “segue”. Esse resultado indica baixa formação de triângulos e poucas comunidades densamente conectadas, sugerindo que as relações de colaboração e seguimento no conjunto de dados são predominantemente dispersas. Apesar disso, a existência de hubs altamente conectados contribui para manter a rede inteiramente conectada e reduz as distâncias entre diferentes regiões do grafo, beneficiando o cálculo de métricas de proximidade utilizadas posteriormente no sistema de recomendação.

3.2.3 Descrição de características e modelagem de texto

Para alimentar a recomendação, foi construído um módulo de engenharia de características. Para cada par (u, r) composto por um usuário u e um repositório candidato r , são extraídas métricas brutas dos nós além das propriedades relacionais descritas na Tabela 2.

Tabela 2: Características relacionais primárias do par usuário-repositório.

Característica	Origem	O que mede
text_similarity	<i>README</i> do perfil e do repositório	Similaridade de cosseno entre vetores <i>TF-IDF</i> dos dois textos
language_match	Linguagens do usuário e do repositório	Coefficiente de Jaccard entre os dois conjuntos de linguagens
follows_owner	Grafo de relações “segue”	Indica se o usuário segue o dono do repositório (0/1)
common_contributors_following	Grafo “segue” e “contribui”	Quantos contribuidores do repositório o usuário já segue
shared_repos_with_owner	Relações “possui” e “contribui”	Em quantos outros repositórios usuário e dono já apareceram juntos
graph_proximity	Grafo completo	$1/(1+d)$, sendo d a distância em arestas no grafo não dirigido
repo_contributor_count	Arestas “contribui”	Número de contribuidores distintos registrados no repositório
repo_freshness	Data do último <i>push</i>	$1/(1 + \Delta t)$, favorecendo repositórios com atividade recente
repo_is_fork	Metadados do repositório	Indica se o repositório é um <i>fork</i> (0/1)

Além dessas características relacionais, o vetor de entrada de cada par inclui atributos absolutos do usuário (razão de *pull requests* sobre eventos totais, escore de atividade, influência na rede, idade da conta, número de linguagens, estrelas recebidas) e do repositório (popularidade em escala logarítmica, tamanho, número de *issues* abertas, presença de *README*).

A característica textual (*text_similarity*) foi implementada com vetorização *TF-IDF* (*Term Frequency-Inverse Document Frequency*) com suavização sub-linear, aplicada sobre um corpus que combina o *README* de perfil pessoal do desenvolvedor com a bio do usuário (do lado do usuário) e o *README*, a descrição e os tópicos do repositório (do lado do repositório). Essa abordagem amplia a cobertura textual em relação ao uso exclusivo dos *READMEs*, mitigando parcialmente a ausência frequente de perfis pessoais detalhados. A similaridade final é calculada pelo produto interno dos vetores *TF-IDF* normalizados (cosseno).

3.2.4 Recomendação de conteúdo

Construção dos rótulos. O rótulo positivo ($y = 1$) de um par (u, r) é definido pela presença de uma aresta de contribuição entre u e r na base coletada. Repositórios em que o próprio usuário

é o criador foram excluídos do conjunto de positivos, pois o GitHub lista automaticamente o dono entre os contribuidores, contar essa relação tornaria a tarefa trivial e não corresponderia ao objetivo de prever contribuições em projetos de terceiros.

Como a API não fornece uma lista de “não-contribuições”, os exemplos negativos ($y = 0$) foram obtidos por amostragem combinando duas estratégias: (i) *negativos difíceis*, repositórios pertencentes a pessoas que o usuário já segue mas nos quais nunca contribuiu; e (ii) *negativos aleatórios*, repositórios sorteados do restante da base. A inclusão de negativos difíceis evita que o modelo aprenda apenas a distinguir contextos temáticos muito distintos, forçando o uso de sinais mais finos. A proporção adotada foi de quatro negativos para cada positivo, com metade categorizados como difíceis.

Divisão treino/teste e modelo. A divisão entre conjuntos de treino e teste foi realizada por usuário (`GroupShuffleSplit` com proporção 75%/25%), garantindo que nenhum desenvolvedor presente no treino apareça no teste. Essa escolha evita que o modelo seja avaliado em exemplos de usuários cujo comportamento ele já observou parcialmente, o que produziria estimativas de desempenho otimistas.

O classificador principal utilizado foi o *Random Forest*, um modelo de *ensemble* de árvores de decisão. Tal modelo foi escolhido porque a matriz de características consolida variáveis vastamente heterogêneas, binários, contagens absolutas, similaridades percentuais e métricas topológicas de grafo, cenário em que árvores de decisão superam modelos lineares sem exigir normalização agressiva. Uma regressão logística foi treinada em paralelo como *baseline* de comparação. O desbalanceamento entre as classes foi tratado ajustando o peso da classe positiva (`class_weight=“balanced”`).

Resultados quantitativos. Os resultados obtidos nos conjuntos de teste estão consolidados na Tabela 3.

Tabela 3: Desempenho dos classificadores no conjunto de teste (3.952 exemplos).

Modelo	Precisão (0)	Revocação (0)	Precisão (1)	Revocação (1)	ROC-AUC	PR-AUC
Random Forest	0,972	0,799	0,581	0,923	0,923	0,746
Reg. Logística (base)	0,917	0,766	0,500	0,771	0,843	0,578

O *Random Forest* obteve ROC-AUC de 0,923 e PR-AUC de 0,746, superando o *baseline* de regressão logística (ROC-AUC 0,843; PR-AUC 0,578) em todas as métricas relevantes. A revocação elevada na classe positiva (0,923) indica que o modelo é eficaz em identificar os pares que de fato resultarão em contribuição, ao custo de uma precisão mais modesta (0,581) — comportamento desejável em um sistema de recomendação, onde recuperar as oportunidades relevantes é mais importante do que garantir que todos os candidatos recomendados sejam perfeitos.

Importância das características. A Figura 4 apresenta as 10 características de maior importância segundo o *Random Forest*.

Tabela 4: Importância das 10 principais características (Random Forest).

Característica	Importância
repo_contributor_count	27,4%
follows_owner	20,7%
repo_is_fork	16,1%
graph_proximity	10,1%
repo_size_kb	7,0%
repo_freshness	6,1%
owner_follower_count	2,5%
language_match	1,5%
common_contributors_following	1,3%
text_similarity	1,2%

O número de contribuidores existentes no repositório (`repo_contributor_count`, 27,4%) emergiu como o sinal mais preditivo, sugerindo que projetos com ecossistemas de contribuição já estabelecidos são mais propensos a atrair novos colaboradores. A variável `follows_owner` (20,7%) confirma que a ponte social direta criador-colaborador é um forte combustível para o engajamento. A natureza derivativa do repositório (`repo_is_fork`, 16,1%) indica que *forks* concentram atividade de contribuição de forma distinta dos projetos originais. Em contrapartida, características de popularidade nominal dos usuários (número de seguidores, presença de blog, *README* de perfil) tiveram importância marginal, evidenciando que conexões concretas na rede superaram métricas de vaidade de perfil como preditores de comportamento colaborativo.

Estudo de caso: usuário luccastera. Para avaliar o funcionamento em contexto real, o modelo foi aplicado ao desenvolvedor `luccastera`, produzindo as recomendações exibidas na Figura 2. Os repositórios recomendados com maior pontuação (0,880 a 0,860) concentram-se em Ruby e JavaScript, linguagens consistentes com o histórico do usuário. A `graph_proximity` manteve-se em 0,200 a 0,250 para todos os candidatos (distância de 4 a 3 arestas no grafo), sinalizando que o usuário está a poucos graus de separação dos criadores desses projetos. A variável `text_similarity`, embora presente com valores modestos (0,006 a 0,022), contribuiu marginalmente para a diferenciação entre candidatos próximos em pontuação.

Top 10 recomendações para 'luccastera'

```
#01 thinkdry/jekyll
Linguagem: Ruby ★ Estrelas: 2
Descrição: Jekyll is a blog-aware, static site generator in Ruby
Tópicos: nan
Sinais: text similarity: 0.013 | graph proximity: 0.250
Score: 0.8805

#02 indmill/homebrew
Linguagem: Ruby ★ Estrelas: 1
Descrição: The missing package manager for OS X
Tópicos: nan
Sinais: text similarity: 0.022 | graph proximity: 0.200
Score: 0.8715

#03 mitsimac/homebrew
Linguagem: Ruby ★ Estrelas: 1
Descrição: The missing package manager for OS X.
Tópicos: nan
Sinais: text similarity: 0.022 | graph proximity: 0.200
Score: 0.8710

#04 brajadorable/activeresource
Linguagem: Ruby ★ Estrelas: 1
Descrição: nan
Tópicos: nan
Sinais: text similarity: 0.015 | graph proximity: 0.200
Score: 0.8698

#05 jeffp/searchlogic
Linguagem: Ruby ★ Estrelas: 2
Descrição: Searchlogic provides object based searching, common named sc
Tópicos: nan
Sinais: text similarity: 0.006 | language match: 0.125 | graph proximity: 0.250
Score: 0.8698
```

Figura 1: Top 10 recomendações geradas pelo modelo Random Forest para o usuário luccastera. Sinais exibidos: similaridade textual (*text similarity*), compatibilidade de linguagem (*language match*) e proximidade no grafo (*graph proximity*).


```
#06 maxdevel/gitx
  Linguagem: Objective-C ★ Estrelas: 1
  Descrição: A gitk clone for OS X
  Tópicos: nan
  Sinais: text similarity: 0.016 | graph proximity: 0.200
  Score: 0.8695

#07 scrubber/nokogiri
  Linguagem: Ruby ★ Estrelas: 1
  Descrição: Nokogiri (鋸) is an HTML, XML, SAX, and Reader parser with XP
  Tópicos: nan
  Sinais: text similarity: 0.011 | graph proximity: 0.200
  Score: 0.8680

#08 krsmurata/JavaScript-Garden
  Linguagem: JavaScript ★ Estrelas: 2
  Descrição: A collection of documentation about the most quirky parts of
  Tópicos: nan
  Sinais: text similarity: 0.018 | language match: 0.125 | graph proximity: 0.200
  Score: 0.8661

#09 angelic/dm-core
  Linguagem: Ruby ★ Estrelas: 2
  Descrição: DataMapper - Core
  Tópicos: nan
  Sinais: text similarity: 0.008 | language match: 0.125 | graph proximity: 0.200
  Score: 0.8650

#10 angelic/paperclip
  Linguagem: Ruby ★ Estrelas: 1
  Descrição: Easy file attachment management for ActiveRecord
  Tópicos: nan
  Sinais: text similarity: 0.008 | language match: 0.125 | graph proximity: 0.200
  Score: 0.8614
```

Figura 2: Top 10 recomendações geradas pelo modelo Random Forest para o usuário luccastera. Sinais exibidos: similaridade textual (*text similarity*), compatibilidade de linguagem (*language match*) e proximidade no grafo (*graph proximity*).

4 Discussão

Os resultados obtidos oferecem uma leitura coerente com a natureza do ecossistema de colaboração em código aberto. A predominância de sinais relacionais (*repo_contributor_count*, *follows_owner*, *graph_proximity*) sobre atributos textuais e de perfil confirma que os desenvolvedores contribuem primariamente dentro de suas “bolhas” sociais, guiados pelas conexões que já possuem e pelos projetos que circulam em seu círculo próximo, e não pela simbiose entre descrições textuais de perfil e *READMEs* de repositórios.

A anulação prática de *text_similarity* na maioria dos casos não implica que o conteúdo semântico seja irrelevante para a colaboração; reflete antes uma limitação dos dados: grande parte dos desenvolvedores não mantém *README* de perfil pessoal no GitHub, e a esparsidade resultante impede que o modelo extraia sinal textual robusto. Em um cenário com cobertura

textual plena, a similaridade semântica teria papel mais relevante.

A alta revocação do *Random Forest* na classe positiva (0,923) é um resultado desejável para o sistema de recomendação: significa que o modelo recupera a grande maioria dos repositórios em que o usuário de fato viria a contribuir. A precisão mais baixa (0,581) implica que parte das recomendações não se concretizaria em contribuição, o que é aceitável em sistemas de recomendação, onde o objetivo é apresentar opções relevantes e não predizer com certeza absoluta.

A natureza estática do grafo representa a principal limitação metodológica: todas as relações são tratadas como simultâneas, sem levar em conta a ordem temporal em que foram formadas. Isso impede, por exemplo, distinguir entre um usuário que passou a seguir o criador de um projeto antes ou depois de contribuir nele, informação que seria valiosa para o aprendizado de causalidade.

5 Conclusões

Este trabalho demonstrou a viabilidade de construir um sistema de recomendação de repositórios no ecossistema do GitHub por meio da combinação de características do desenvolvedor, propriedades dos projetos e métricas extraídas de um grafo heterogêneo com mais de 137 mil nós e 210 mil arestas. O classificador *Random Forest* obteve ROC-AUC de 0,923 e PR-AUC de 0,746, superando consistentemente o *baseline* linear. Identificou-se que interações diretas na rede (seguir criadores, pertencer a ecossistemas de contribuição consolidados) possuem muito mais peso preditivo do que atributos de popularidade nominal ou afinidade textual superficial.

No que tange às limitações, a exploração semântica com *TF-IDF* mostrou-se ineficaz pelo fato concreto de que grande parcela dos usuários não mantém descrições pessoais extensas. Além disso, a modelagem operou de maneira estática, sem incorporar a cronologia das interações.

Como continuções naturais do trabalho, destacam-se: (i) a introdução de grafos temporais, em que cada aresta obedece à cronologia das interações, permitindo modelar ciclos de engajamento e decaimento de projetos; (ii) o uso de modelos de linguagem pré-treinados para representação textual dos *READMEs* em substituição ao *TF-IDF*, aumentando a robustez semântica mesmo com textos curtos; e (iii) a adoção de embutimentos vetoriais de grafos completos via *Node2Vec* ou redes neurais em grafo (GNNs), capazes de aprender representações estruturais latentes que transcendem a engenharia manual de características topológicas.